COS40007 – Artificial Intelligence Engineering

Applied Project Task 2 (Group Task 2)

Implement a GitHub Actions Trigger

Project: Anti-Spoofing for Secure Login – Edge Liveness Detection on Raspberry Pi

Item	Details
Unit	COS40007 Artificial Intelligence Engineering
Task	Group Task 2 – Studio Activity 7 (Week 7)
Group No.	3
Group members	Dinh Phu Nguyen – 105027282; Phuc Lam Nguyen – 105006540
Submission date	11/08/2026
GitHub repository	https://github.com/phu1009/COS40007---AI-Engineering-

Pipeline overview

We added a basic machine-learning pipeline to our group repository with GitHub Actions. Every push to **main** (and every pull request to **main**, or a manual run) starts a GitHub-hosted **ubuntu-latest** runner that installs our dependencies, runs our project training script **model.py**, and uploads the results as downloadable artefacts.

```
git push  (or pull request to main / Run workflow button)
    ↓
GitHub Actions triggered  →  workflow 'Train ML Model' (.github/workflows/train.yml)
    ↓
ubuntu-latest runner spins up
    ↓
checkout code → set up Python 3.11 → cache pip → install requirements.txt
    ↓
python model.py
    → builds a live / print-attack / replay-attack face dataset, subject-disjoint split
    → trains a lightweight CNN anti-spoofing (PAD) model
    → saves model_results.png, metrics.txt and pad_model.keras
    ↓
upload artefact 'model-results'  (downloadable from the Actions UI)
```

model.py is our project file. It trains the face presentation-attack-detection model from our Task 1 blueprint: a small depthwise-separable CNN (MiniFASNet-style, light enough for a Raspberry Pi) that outputs a live score. The threshold is chosen on validation data (BPCER ≤ 5 %) and the test set is scored with the ISO/IEC 30107-3 metrics APCER, BPCER and ACER, plus accuracy and ROC-AUC. The real datasets (CelebA-Spoof, Replay-Attack) are licensed and tens of GB, so they cannot be downloaded inside CI; the script therefore generates a small synthetic stand-in with the same three classes. Training takes about 30 seconds and the whole run about one minute, which keeps the pipeline simple, as the brief asks.

1. Repository & File Structure

The repository contains exactly the required structure at the root of the group repository:

```
COS40007---AI-Engineering-/          (group repository)
├── .github/
│   └── workflows/
│       └── train.yml                ← pipeline definition
├── model.py                         ← project training script
├── requirements.txt                 ← dependencies
├── README.md                        ← pipeline description + status badge
├── Group Project/                   ← Task 1 documents and project code
└── Portfolio Assessment 1–3/
```

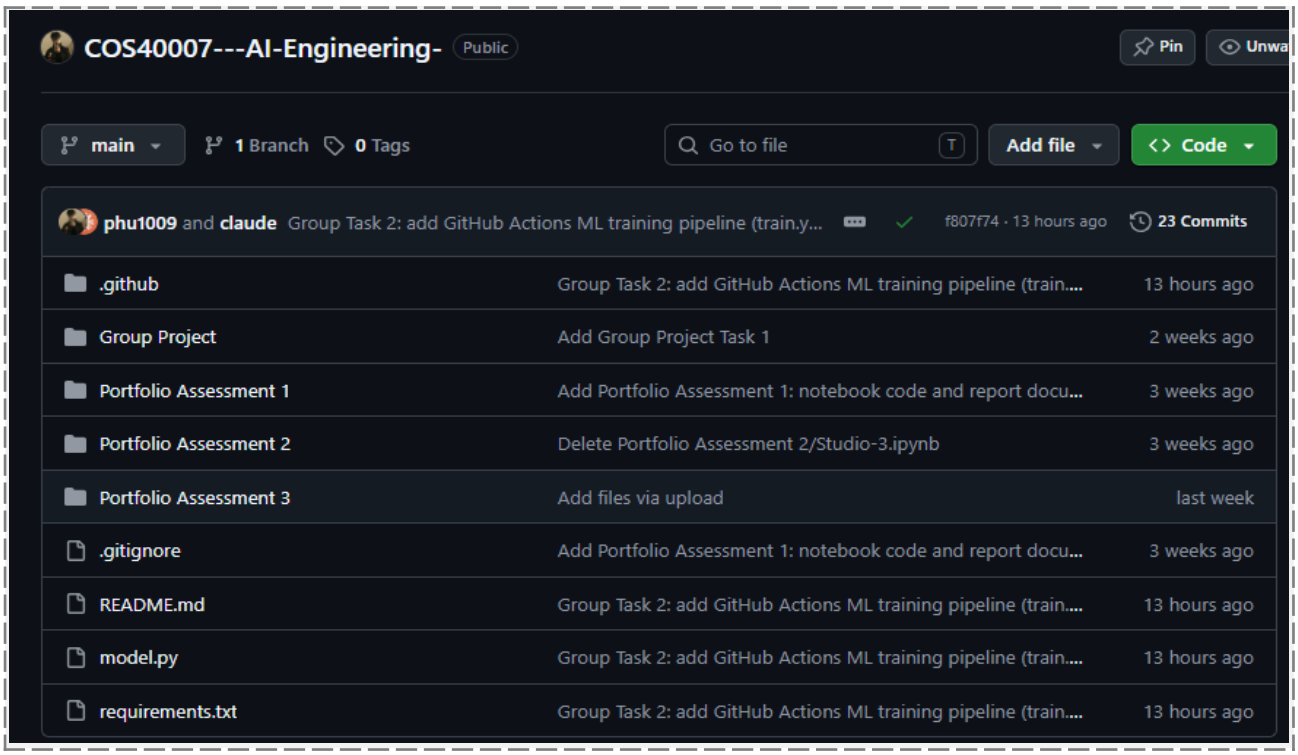


Figure 1. Repository root on GitHub showing .github, model.py, requirements.txt and README.md (https://github.com/phu1009/COS40007---AI-Engineering-).

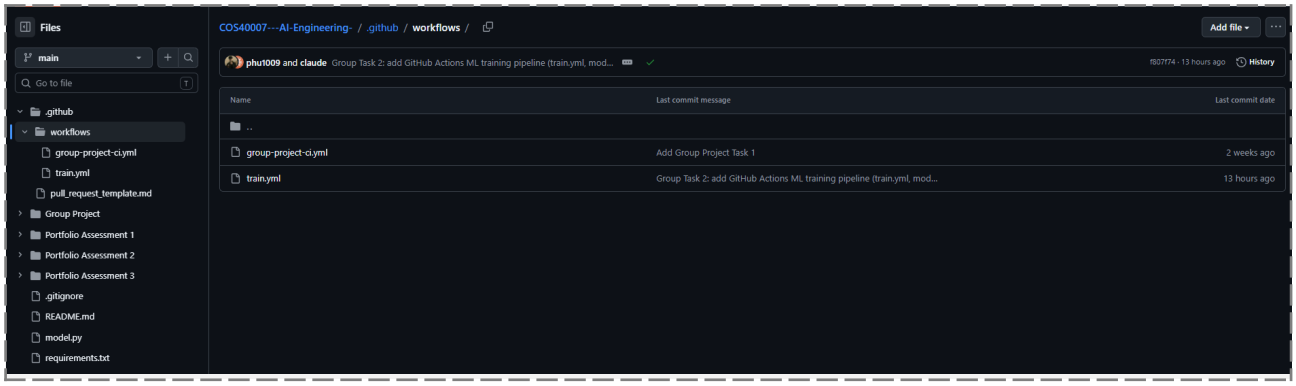


Figure 2. The workflow file in the exact required location .github/workflows/train.yml (https://github.com/phu1009/COS40007---AI-Engineering-/tree/main/.github/workflows).

2. requirements.txt

```
tensorflow==2.21.0
numpy==2.4.6
matplotlib==3.11.2
```

- All three required libraries are listed with correct package names: **tensorflow**, **numpy**, **matplotlib**.
- Versions are pinned so every run installs the same environment: TensorFlow 2.21.0 (the version given in the Studio guide) supports the runner's Python 3.11, and the NumPy and Matplotlib versions were installed and tested together with it (pip check: no broken requirements).
- Changing this file also changes the pip cache key in train.yml, so the cache is refreshed automatically.

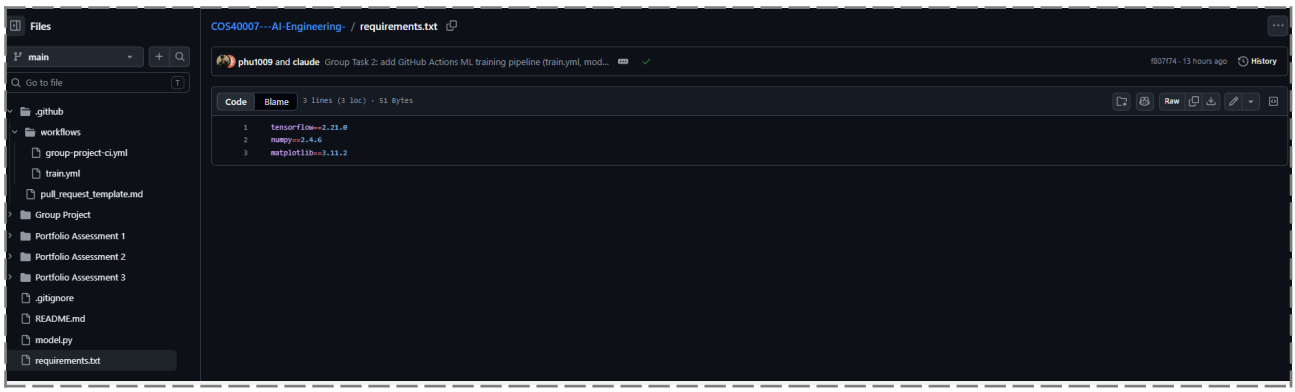


Figure 3. requirements.txt in the repository root (https://github.com/phu1009/COS40007---AI-Engineering-/blob/main/requirements.txt).

3. Workflow Triggers (on:)

```yaml
name: Train ML Model

on:
  push:
    branches: [ main ]        # runs every time we push to main
  pull_request:
    branches: [ main ]        # runs on pull requests to main
  workflow_dispatch:          # allows a manual run from the GitHub Actions UI
```

Trigger	When the pipeline runs
push → branches: [main]	Every time a commit is pushed to main.
pull_request → branches: [main]	Every time a pull request targeting main is opened or updated, so changes are tested before merging.
workflow_dispatch	Manual run from the GitHub UI: Actions → Train ML Model → Run workflow.

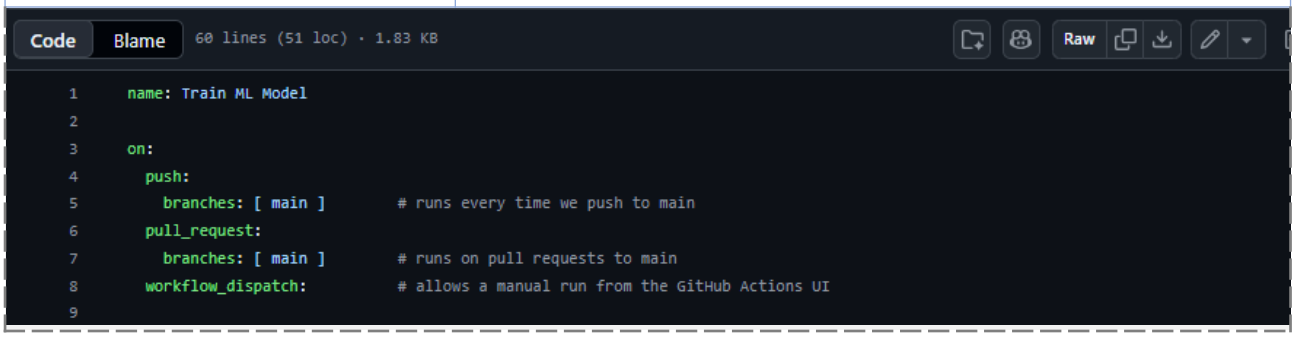


Figure 4. train.yml – workflow name and the three triggers (https://github.com/phu1009/COS40007---AI-Engineering-/blob/main/.github/workflows/train.yml).

4. Pipeline Job Definition

The workflow defines one job named **train** that runs on the GitHub-hosted **ubuntu-latest** runner and executes the required steps in order:

#	Step	What it does
1	Checkout code	actions/checkout@v4 – copies the repository onto the runner.
2	Set up Python	actions/setup-python@v5 – installs Python 3.11.
3	Cache pip dependencies	actions/cache@v4 – caches ~/.cache/pip, keyed on the hash of requirements.txt, so repeat runs install faster.

#	Step	What it does
4	Install dependencies	pip install -r requirements.txt
5	Train model	python model.py – trains the anti-spoofing model and writes the result files.
6	Upload results	actions/upload-artifact@v4 – uploads model_results.png, metrics.txt and pad_model.keras as the artefact 'model-results'; fails the run if a file is missing.
7	Show metrics in run summary	Extra step: prints metrics.txt on the run's summary page.

```
jobs:
  train:
    runs-on: ubuntu-latest    # GitHub-hosted Linux runner (free)

    steps:
      # 1. Check out the repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Set up Python
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      # 3. Cache pip packages so repeat runs are faster
      - name: Cache pip dependencies
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-

      # 4. Install dependencies from requirements.txt
      - name: Install dependencies
        run: pip install -r requirements.txt

      # 5. Train the anti-spoofing (PAD) model - runs model.py
      - name: Train model
        run: python model.py

      # 6. Upload the plot, metrics and trained model as artefacts
      #    so they can be downloaded from the GitHub Actions UI
      - name: Upload results
        uses: actions/upload-artifact@v4
        with:
          name: model-results
          path: |
            model_results.png
            metrics.txt
            pad_model.keras
          if-no-files-found: error

      # 7. Show the metrics on the run's summary page
      - name: Show metrics in run summary
        run: |
          echo '### Group 3 - Anti-Spoofing model metrics' >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
          cat metrics.txt >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
```

Code Blame 60 lines (51 loc) · 1.83 KB Raw

```yaml
jobs:
  train:
    runs-on: ubuntu-latest    # GitHub-hosted Linux runner (free)

    steps:
      # 1. Check out the repository code
      - name: Checkout code
        uses: actions/checkout@v4

      # 2. Set up Python
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      # 3. Cache pip packages so repeat runs are faster
      - name: Cache pip dependencies
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-${{ hashFiles('requirements.txt') }}
          restore-keys: |
            ${{ runner.os }}-pip-

      # 4. Install dependencies from requirements.txt
      - name: Install dependencies
        run: pip install -r requirements.txt

      # 5. Train the anti-spoofing (PAD) model - runs model.py
      - name: Train model
        run: python model.py

      # 6. Upload the plot, metrics and trained model as artefacts
      #    so they can be downloaded from the GitHub Actions UI
      - name: Upload results
        uses: actions/upload-artifact@v4
        with:
          name: model-results
          path: |
            model_results.png
            metrics.txt
            pad_model.keras
          if-no-files-found: error

      # 7. Show the metrics on the run's summary page
      - name: Show metrics in run summary
        run: |
          echo '### Group 3 - Anti-Spoofing model metrics' >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
          cat metrics.txt >> "$GITHUB_STEP_SUMMARY"
```

Figure 5. train.yml – job 'train' on ubuntu-latest with the sequential steps.

5. Successful Pipeline Execution

The pipeline ran to completion without errors on all four runs: runs #1, #2 and #4 were triggered by git push and run #3 was started manually (workflow_dispatch); each took about one minute (Figure 11). Figures 6–7 show run #3. Every step – checkout, Python set-up, cache, install, training and upload – shows a green check mark. The only annotation is a GitHub platform warning that actions built for Node.js 20 now run on Node.js 24; it does not affect the run, and we keep actions/upload-artifact@v4 as specified.

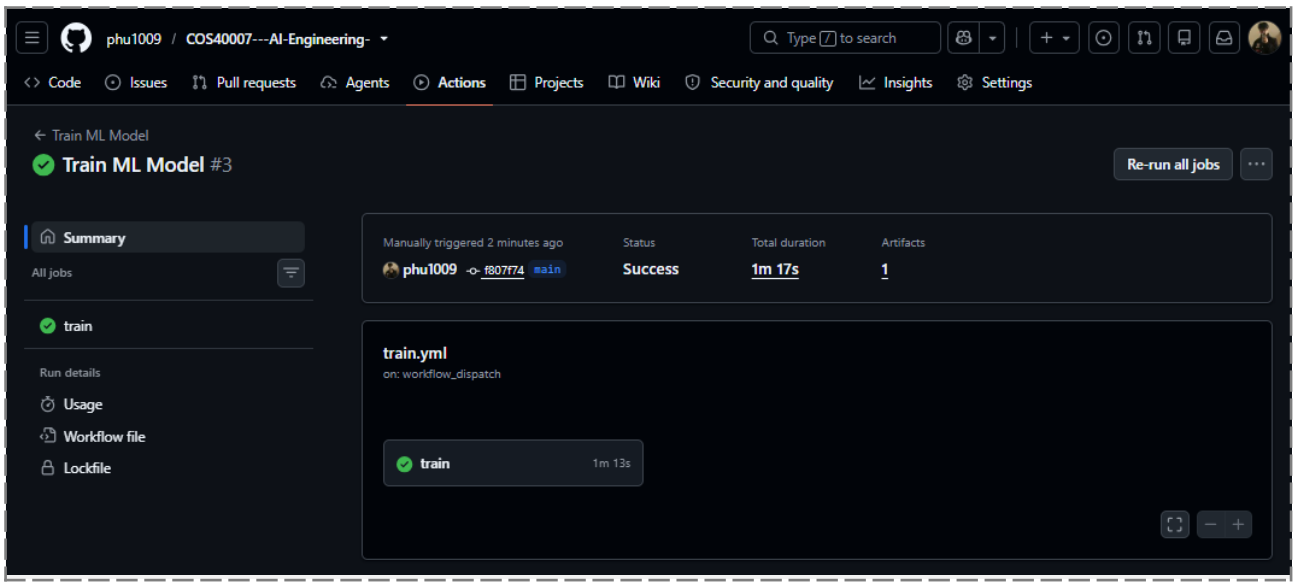


Figure 6. Run summary: status Success, job 'train' passed, artefact listed (https://github.com/phu1009/COS40007---AI-Engineering-/actions/runs/42494208816).

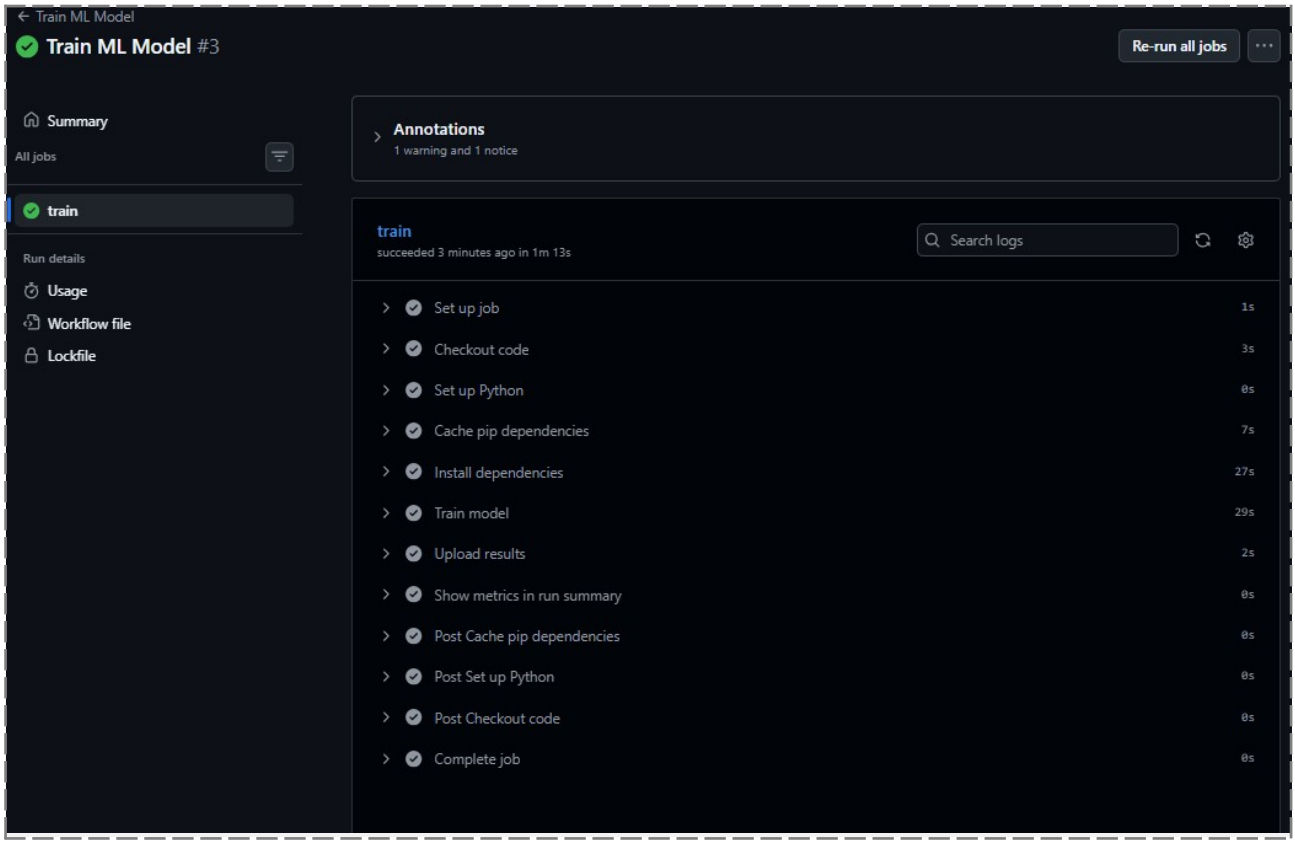


Figure 7. Job log: all steps of job 'train' completed with green check marks (https://github.com/phu1009/COS40007---AI-Engineering-/actions/runs/42494208816/job/128772869873).

6. Artefact Generation & Upload

model.py generates the results files and the workflow uploads them with **actions/upload-artifact@v4** under the name **model-results**:

File	Content
model_results.png	Six panels: test samples (live / print / replay), loss and ROC-AUC per epoch, ROC curve with the chosen operating point, score distribution per class,

File	Content
	confusion matrix.
metrics.txt	Run number, commit, TensorFlow version, data split, threshold, APCER per attack type, BPCER, ACER, accuracy and ROC-AUC.
pad_model.keras	The trained model, ready for later conversion to ONNX/TFLite for the Raspberry Pi.

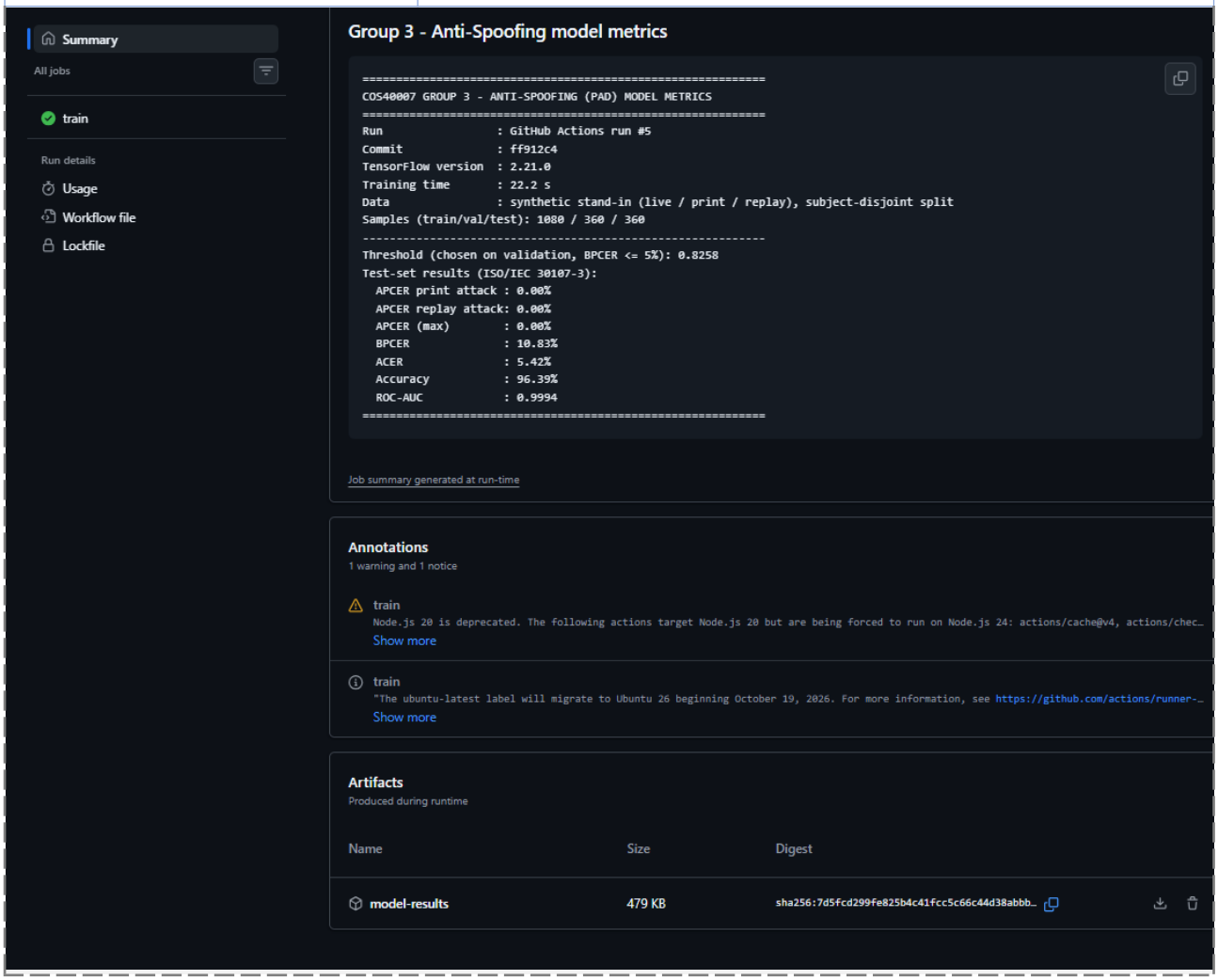


Figure 8. Artifacts section of the run: 'model-results' available for download.

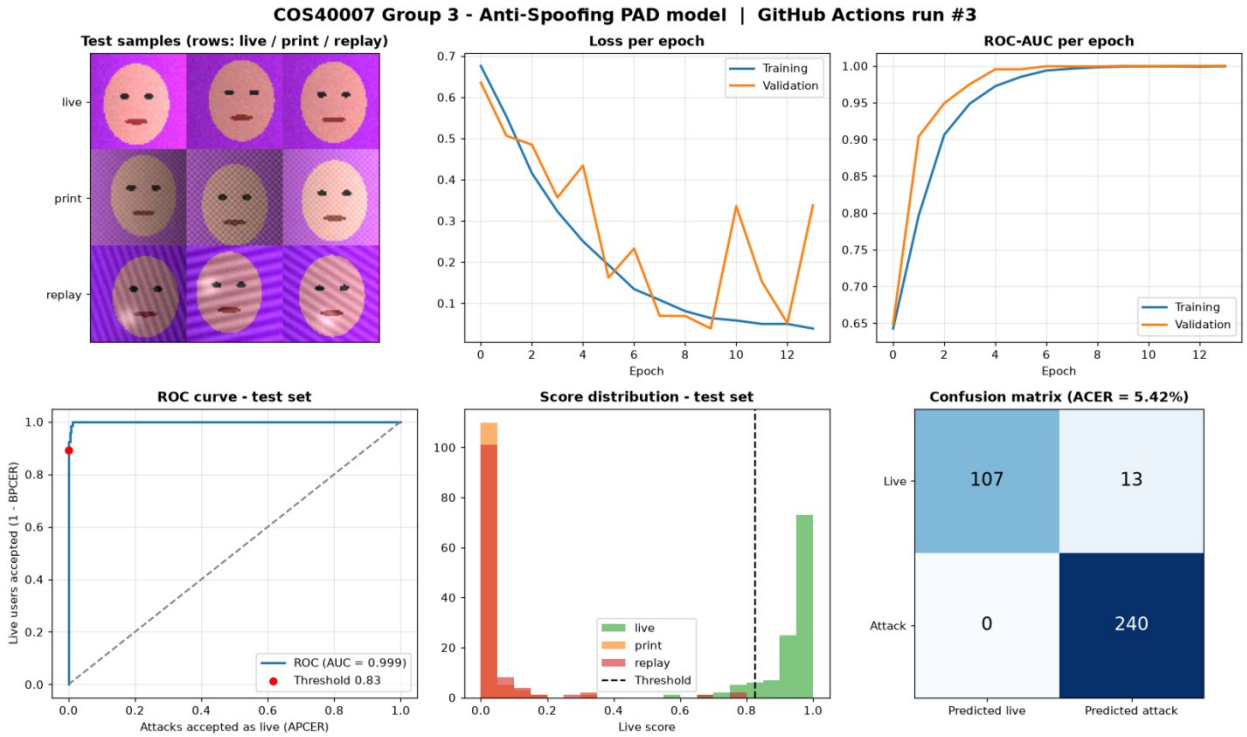


Figure 9. Downloaded artefact: model_results.png produced by the pipeline.

Content of the downloaded metrics.txt:

```
============================================================
COS40007 GROUP 3 - ANTI-SPOOFING (PAD) MODEL METRICS
============================================================
Run                 : GitHub Actions run #3
Commit              : f807f74
TensorFlow version  : 2.21.0
Training time       : 22.8 s
Data                : synthetic stand-in (live / print / replay), subject-disjoint split
Samples (train/val/test): 1080 / 360 / 360
------------------------------------------------------------
Threshold (chosen on validation, BPCER <= 5%): 0.8258
Test-set results (ISO/IEC 30107-3):
  APCER print attack : 0.00%
  APCER replay attack: 0.00%
  APCER (max)        : 0.00%
  BPCER              : 10.83%
  ACER               : 5.42%
  Accuracy           : 96.39%
  ROC-AUC            : 0.9994
============================================================
```

7. Demonstration of Functionality

We demonstrated the full cycle: trigger the pipeline with git push, find the run in the GitHub Actions UI, and download the artefacts.

7.1 Trigger with git push

```
PS D:\University\COS40007---AI-Engineering->
PS D:\University\COS40007---AI-Engineering-> git add README.md
PS D:\University\COS40007---AI-Engineering-> git commit -m "Group 3: trigger training pipeline
(demo)"
[main bfde76f] Group 3: trigger training pipeline (demo)
 1 file changed, 16 insertions(+), 16 deletions(-)
PS D:\University\COS40007---AI-Engineering-> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 560 bytes | 560.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/phu1009/COS40007---AI-Engineering-
   f807f74..bfde76f  main -> main
PS D:\University\COS40007---AI-Engineering->
```

Figure 10. Terminal: commit and git push to main, which triggers the workflow.

7.2 Locate the run in the GitHub Actions UI

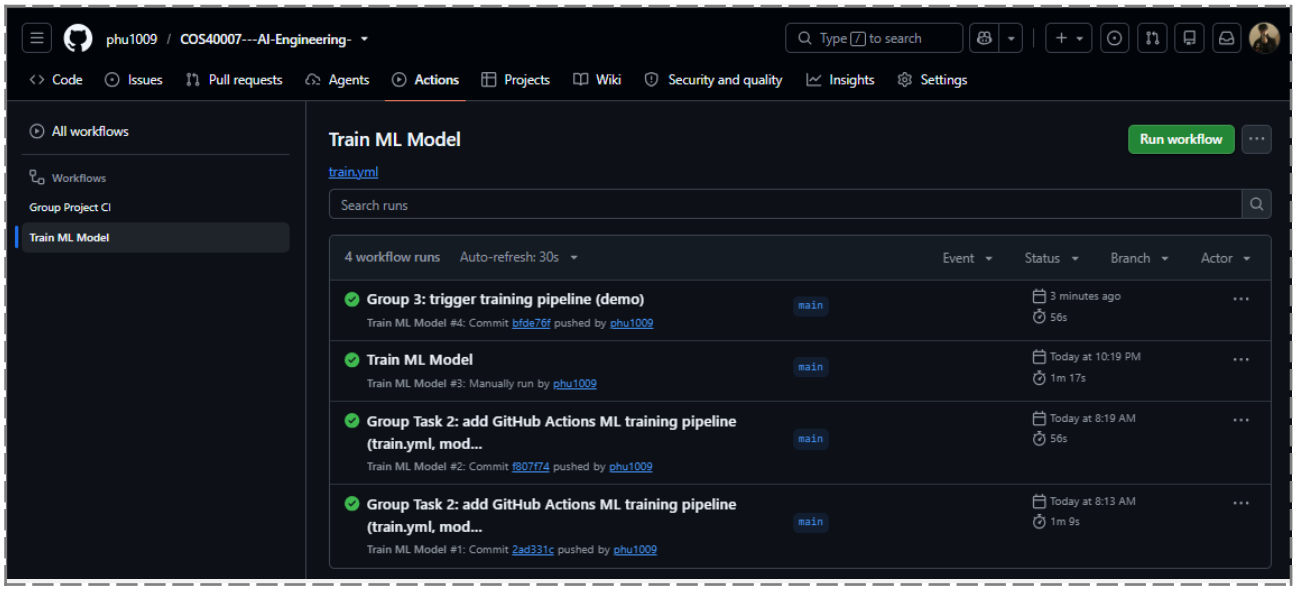


Figure 11. Actions tab → 'Train ML Model': the runs started by push (and by the manual trigger) (https://github.com/phu1009/COS40007---AI-Engineering-/actions/workflows/train.yml).

7.3 Manual trigger (workflow_dispatch)

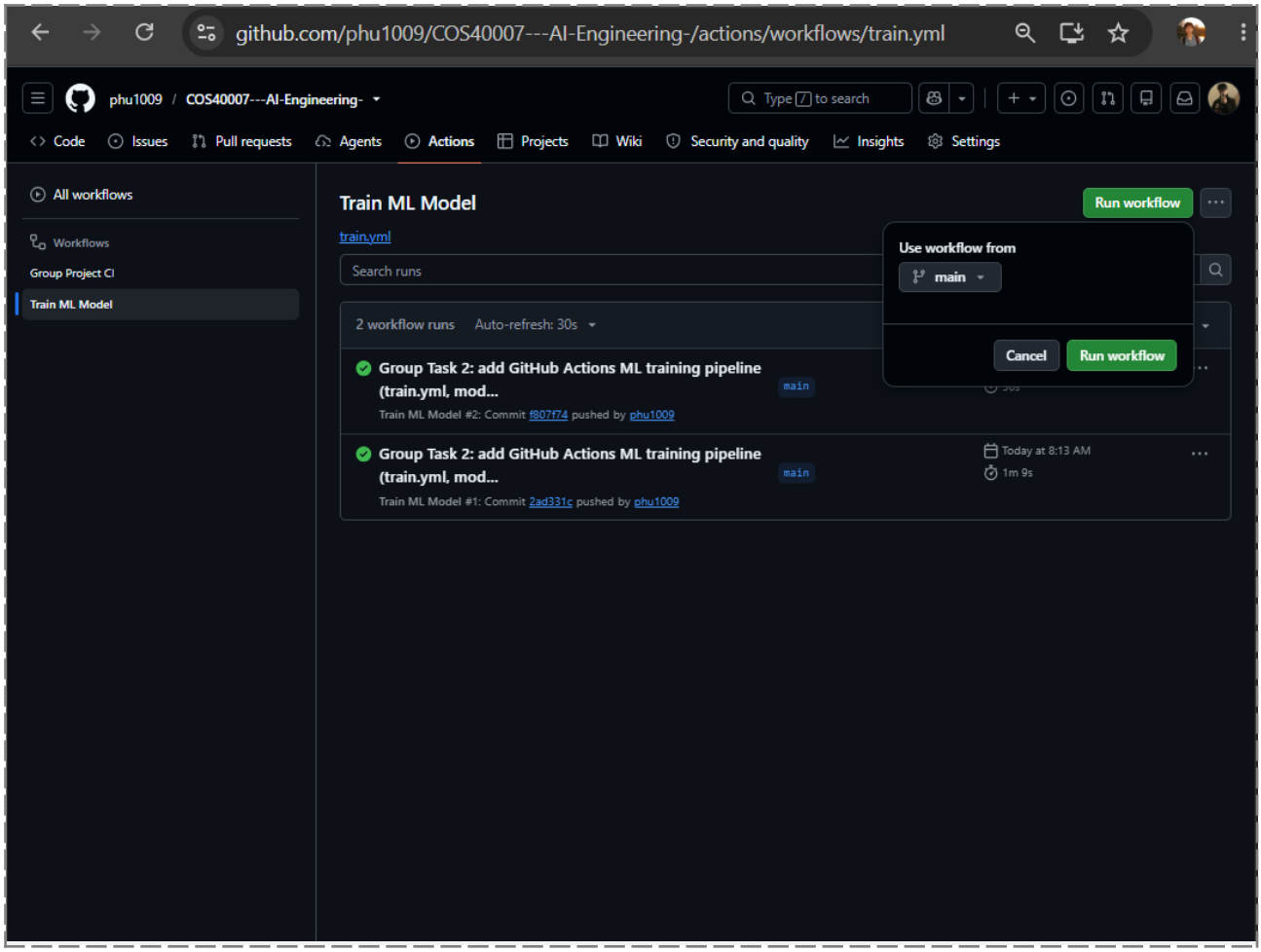


Figure 12. Run workflow button on the 'Train ML Model' page, used to start the pipeline manually.

7.4 Download the artefacts

Open the finished run → scroll to **Artifacts** → click **model-results** to download a zip containing model_results.png, metrics.txt and pad_model.keras (Figures 8–9 show the downloaded files).

8. GitHub Link

Group repository:

https://github.com/phu1009/COS40007---AI-Engineering-

Workflow (Actions):

https://github.com/phu1009/COS40007---AI-Engineering-/actions/workflows/train.yml

Workflow file:

https://github.com/phu1009/COS40007---AI-Engineering-/blob/main/.github/workflows/train.yml